LND Learning & Development Platform

Project & Architecture Overview

Audience: Management · Engineering Management · Development Team
Phase: Proof of Concept | Status: Build green, 52/52 tests passing, web demo runnable | 2026-10-28

1. Executive summary

The LND Learning & Development Platform is a proof of concept for an internal training portal. It lets an organization publish training videos, organize them into learning paths, assign training to audiences, assess learners with quizzes, and track completion — for three kinds of users: **L&D Admin**, **Employee**, and **Partner**.

This phase deliberately has **no backend and no database**. It runs entirely in the browser using bundled sample data and the browser's local storage. The purpose is to validate the product experience, information architecture, and UI direction quickly and cheaply, before committing to backend, infrastructure, and integration spend.

What this delivers today

- A working, demo-able application covering all twelve target capabilities.
- A login screen with three demo roles and role-appropriate access.
- A mobile-first interface in the Safexpress visual identity, buildable toward a native Android app via Capacitor.
- A clean, documented, test-backed codebase that a team can extend.

What this is not (by design, this phase)

- Not connected to any real system (no SSO, Slack, vendor apps, or AWS).
- Not using real authentication — the login is a demo gate, not secure sign-in.
- Not persisting data on a server — all state is local to the browser/device.

> **Headline recommendation:** the POC is suitable for stakeholder demos and user feedback now. Converting it into a production system is a well-understood next phase (backend, real auth, real data), and the current architecture was built to make that transition straightforward rather than a rewrite.

2. Objectives and scope

2.1 Business objectives

- Demonstrate the end-to-end L&D journey (discover → learn → assess → track) without backend investment.
- Provide a tangible artifact for stakeholder feedback and go/no-go decisions.

- De-risk the UI, navigation, and role model before production build-out.

2.2 In scope (this phase)

- Dashboard with project milestones and a personal learning snapshot.
- Video catalogue with search, filtering, and categories.
- Video metadata management for L&D admins (create/edit/delete).
- Video playback with progress tracking and resume.
- Training assignments targeted at a simulated audience (role, group, individual).
- My Learning page (assigned / in-progress / completed).
- Quizzes: multiple choice and true/false, with configurable pass marks.
- Learning paths where later steps unlock after earlier assessments are passed.
- Participant history and completion reports (admin).
- Feedback capture on videos and paths.
- Demo login with three roles and role-based access.
- Responsive, mobile-first interface; Android-ready via Capacitor.

2.3 Explicitly out of scope (this phase)

- Real database or persistence server.
- Backend API and business logic on the server.
- Real identity provider / SSO, Slack, or third-party vendor integration.
- Any requirement for AWS credentials to run the demo.

3. Solution architecture

3.1 Architectural style

A **single-page application (SPA)** built with **Angular 17** using the classic, strict **NgModule** architecture (no standalone components). UI is built with **Ionic 7** components and themed with SCSS/CSS variables. **Capacitor 6** wraps the web build for native packaging (Android configured). The app is fully client-side: seed data loads from bundled JSON fixtures, and all mutable demo state is stored in the browser's `localStorage`. There is no network dependency to run the demo.

3.2 Module structure

```
AppModule (bootstrap, IonicModule.forRoot, APP_INITIALIZER)
├── AppRoutingModule      Root routes, lazy loading, route guards
├── CoreModule            Singleton services, guards, domain models (imported once)
├── SharedModule          Reusable UI components + pipes; re-exports IonicModule
├── LayoutModule          App shell: header, collapsible sidenav, footer, user menu
└── Feature modules (lazy-loaded, each with its own routing module)
    ├── AuthModule          Login page (public, outside the shell)
    ├── DashboardModule     Dashboard + milestones
    ├── CatalogueModule     Catalogue, search, categories
    ├── VideoAdminModule    Video metadata CRUD (admin-only)
    ├── PlayerModule        Playback + progress tracking
    ├── AssignmentsModule   Assignments by audience (admin-only)
    ├── MyLearningModule    Personal learning aggregation
    ├── QuizModule          Take quizzes (MCQ, true/false)
    ├── QuizAdminModule     Quiz builder (admin-only)
    ├── LearningPathsModule Paths with assessment-gated unlocking
    ├── ReportsModule       Participant history + reports (admin-only)
    └── FeedbackModule      Feedback capture and averages
```

Design rules enforced: singletons only in CoreModule (guarded against double-import); SharedModule holds only reusable, stateless UI; feature modules never import each other and communicate through Core services; every feature is lazy-loaded (confirmed by the build emitting one chunk per feature).

3.3 Core services

Service	Responsibility
StorageService	Typed, namespaced localStorage access + reset
MockDataService	Loads and caches JSON fixtures
AuthService	Demo login/logout, session persistence and restore
RoleService	Active user/role state (fixed to the signed-in account)
VideoService	Catalogue + admin CRUD (fixtures merged with stored edits)
ProgressService	Per-user playback position, percent, completion (90% threshold)
QuizService	Scoring (MCQ/true-false), pass marks, attempts, quiz CRUD
LearningPathService	Step lock/unlock logic from progress + attempts
AssignmentService	Audience resolution (role/group/individual)
FeedbackService	Submit, list, average rating
ReportService	Completion and attempt reports; dashboard snapshot
AppStateService	Initializes all services at startup; demo reset

3.4 Routing, guards, and access control

- **AuthGuard** protects the entire authenticated area; unauthenticated users are redirected to `/login`.
- **RoleGuard** restricts `/admin/*` routes to L&D Admin — enforced by navigation and direct URL.
- The shell wraps all guarded routes; `/login` renders outside the shell.
- Role is **fixed to the logged-in account** — no in-app role switching. To change role, sign out and sign in as another user.

4. Technology stack

Layer	Choice	Version	Notes
Framework	Angular	17	Strict NgModule, no standalone
Language	TypeScript	5.4 (strict)	No `any` in domain code
UI	Ionic Framework	7	NgModule mode (IonicModule.forRoot)
Icons	Ionicons	7	Bundled as assets
Reactive	RxJS	7.8	BehaviorSubject-backed state
Styling	SCSS + CSS variables	—	Safexpress green gradient theme
Native shell	Capacitor	6	Android target configured
Unit tests	Jasmine + Karma	5 / 6.4	Headless Chrome

Compatibility: Ionic 7 supports Angular 14+, so Angular 17 with NgModule is fully supported. Capacitor 6 works with the Angular 17 toolchain.

5. Security and data posture

> This is a **demo build**. The following are deliberate POC limitations and **must not be carried into production as-is**:
>
> - **The login is not real authentication.** Demo credentials live in JSON fixtures and are checked in the browser.
> - **No data is protected or encrypted** — all state is in the device's local storage.
> - **No server-side authorization** — role-based access is client-only; production must enforce it on a backend.

6. Current status and verification

- **Build:** production build compiles cleanly; every feature module is a separate lazy-loaded chunk.
- **Tests:** 52 of 52 unit tests pass in headless Chrome (auth, storage, roles, video CRUD, progress/completion, quiz scoring & pass-mark boundaries, path unlocking, assignment resolution, feedback averaging).

- **Runnable demo:** `npm install` then `npm start`; sign in with a demo account. No server, database, or credentials required.

Honest gaps: the native Android project has not been generated yet (`npx cap add android` not run), so it has not been built/launched on a device. UI has not had formal cross-device QA or an accessibility audit.

7. Converting to a native Android application

Configured for Capacitor Android; the native project is not yet created. Remaining steps need the Android SDK and a device/emulator:

```
npm run build          # produce the web build in dist/lnd-app
npx cap add android    # generate the native Android (Gradle) project
npx cap sync           # copy the web build + plugins into the native project
npx cap open android   # open in Android Studio to run on device/emulator
```

Prerequisites: Android Studio + Android SDK, JDK 17. **Before an APK:** app icon/splash, status-bar/safe-area handling, hardware back-button wiring, hosted media (catalogue uses a remote sample video URL), and release signing.

8. Path to production (roadmap)

- **Phase 2 — Backend & real data:** API + database behind the existing service interfaces; move trusted business rules (scoring, unlocking, reporting) to the server.
- **Phase 3 — Real identity & security:** SSO/identity provider; server-enforced authorization; audit logging and data protection.
- **Phase 4 — Integrations:** SSO, Slack, video hosting, AWS as needed.
- **Phase 5 — Hardening & release:** cross-device & accessibility QA, performance budgets, e2e tests, CI/CD, Android (and optionally iOS) store release.

9. Risks and assumptions

Risk / assumption	Impact	Mitigation
Demo login mistaken for real auth	Security misperception	Clearly labeled demo-only; Phase 3 adds real auth
localStorage limits / clearing	Demo data loss	Acceptable for POC; production uses a server
Android not yet built on device	Unknown device issues	Run cap add android + device test early in Phase 2
Remote video URL dependency	Playback needs network	Replace with hosted/owned media in production

10. How to run (developers)

```
npm install
npm start           # dev server at http://localhost:4200
npm run build       # production build
npm test            # unit tests (watch)
ng test --watch=false --browsers=ChromeHeadlessNoSandbox   # headless run
```

Demo accounts (seeded in `src/assets/mock/users.json`, demo-only): L&D Admin `admin / admin123` · Employee `erin / employee123` · Partner `partner / partner123`.
